LOADI 与 LOADK 指令详解

指令概览

指令	作用	数据来源	目标	描述
LOADI	将立即整数放入虚拟栈/寄存器	字节码中的立即数	虚拟栈	A sBx R[A] := sBx
LOADK	将常量池中的值加载到虚拟栈	常量池(K 表)	虚拟栈	A Bx R[A] := K[Bx]

LOADI 指令：加载立即整数

指令功能

LOADI 指令用于将字节码中的**立即整数**加载到虚拟栈/寄存器中。

```
// 指令格式: A sBx  R[A] := sBx
L_OP_LOADI: {
    StkId ra = RA(i);                  // 获取目标寄存器/栈位置
    lua_Integer b = GETARG_sBx(i);     // 解析字节码中的立即数（带符号）
    setivalue(s2v(ra), b);             // 写入栈/寄存器并设置类型
    vmbreak;
}
```

字节码参数解析

Lua 字节码采用按位编码，`LOADI` 使用 sBx 字段存储带符号的立即数：

```
// 指令字段定义
#define SIZE_C 8            // C字段长度
```

```
#define SIZE_B 8            // B字段长度
#define SIZE_Bx (SIZE_C + SIZE_B + 1)   // Bx字段总长度
#define SIZE_OP 7           // 操作码字段长度
#define POS_OP 0            // 操作码位置
#define POS_A (POS_OP + SIZE_OP)        // A参数位置
#define POS_Bx (POS_A + SIZE_A)         // Bx参数位置

// sBx 解码宏
#define GETARG_sBx(i) \
    check_exp(checkopm(i, iAsBx), getarg(i, POS_Bx, SIZE_Bx) - OFFSET_sBx)
```

解码过程:

1. 从指令中提取 Bx 字段（无符号整数）
2. 减去偏移量 `OFFSET_sBx` 得到有符号整数

栈/寄存器写入机制

Lua VM 使用 `TValue` 结构表示栈元素:

```
// 值联合体
typedef union Value {
    lua_Integer i;    // 整数
    lua_Number n;     // 浮点数
    void* p;          // 轻量级用户数据
    GCObject* gc;     // 垃圾回收对象
} Value;

// 类型化值结构
typedef struct TValue {
    Value value_;     // 实际值
    int tt_;          // 类型标记
} TValue;

// 写入整数的宏
#define setivalue(obj, x) \
{                         \
    TValue *io = (obj);   \
    val_(io).i = (x);     \
    settt_(io, LUA_VNUMINT); \
}
```

写入流程：

1. `s2v(ra)` 获取目标位置的 `TValue` 指针
2. `val_(io).i = b` 写入整数值
3. `settt_` 设置类型为 `LUA_VNUMINT`

LOADK 指令：加载常量值

指令功能

LOADK 指令用于将常量池（K 表）中的值加载到虚拟栈/寄存器中。

```
// 指令格式：A Bx  R[A] := K[Bx]
L_OP_LOADK: {
    StkId ra = RA(i);              // 获取目标寄存器/栈位置
    TValue *rb = k + GETARG_Bx(i); // 从常量池获取源值指针
    setobj2s(L, ra, rb);           // 复制值到目标位置
    vmbreak;
}
```

常量池访问

`LOADK` 通过 Bx 索引访问常量池（K 表）：

```
// 常量池访问
TValue *rb = k + GETARG_Bx(i);  // k为常量池基地址

// 对象复制宏
#define setobj2s(L, o1, o2) setobj(L, s2v(o1), o2)
```

访问流程：

1. 从指令中提取 Bx 字段（无符号整数）

2. 以 `k` （常量池基地址）加上 Bx 偏移量得到源值指针
3. 使用 `setobj2s` 将值复制到目标位置

与 LOADI 的关键区别

特性	LOADI	LOADK
数据来源	字节码内部	常量池(K 表)
值类型	仅整数	任意 Lua 类型
编码	有符号 sBx	无符号 Bx
性能	更快（直接编码）	较慢（内存访问）

虚拟栈模型

两个指令都操作 Lua VM 的**虚拟栈**，这是执行引擎的核心数据结构：

```
字节码执行流程：
1. LOADI: [字节码立即数] → [虚拟栈 TValue]
2. LOADK: [常量池值] → [虚拟栈 TValue]

虚拟栈元素：TValue { 类型标记 | 值(整数/浮点数/对象) }
```

总结

1. LOADI 适用于加载**小整型立即数**，值直接编码在指令中，执行效率高
2. LOADK 适用于加载**各种类型的常量**，通过索引访问常量池，灵活性更强
3. 两种指令都体现了 Lua VM 的**栈式虚拟机**设计思想，所有操作都通过栈传递数据

4. 通过 `TValue` 的统一表示，Lua 实现了动态类型的运行时支持